UNIVERSIDAD TÉCNICA DE AMBATO

FACULTAD DE INGENIERÍA EN SISTEMAS, ELECTRÓNICA E INDUSTRIAL

FLUTTER



JULIO 2026-DICIEMBRE 2026

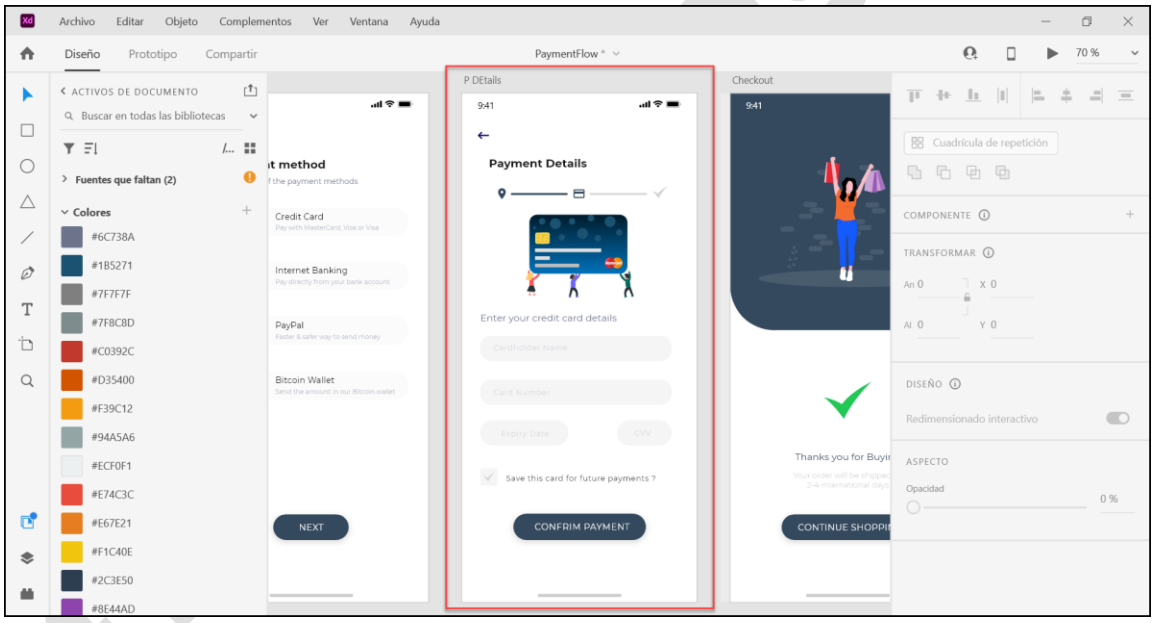
16/10/2026

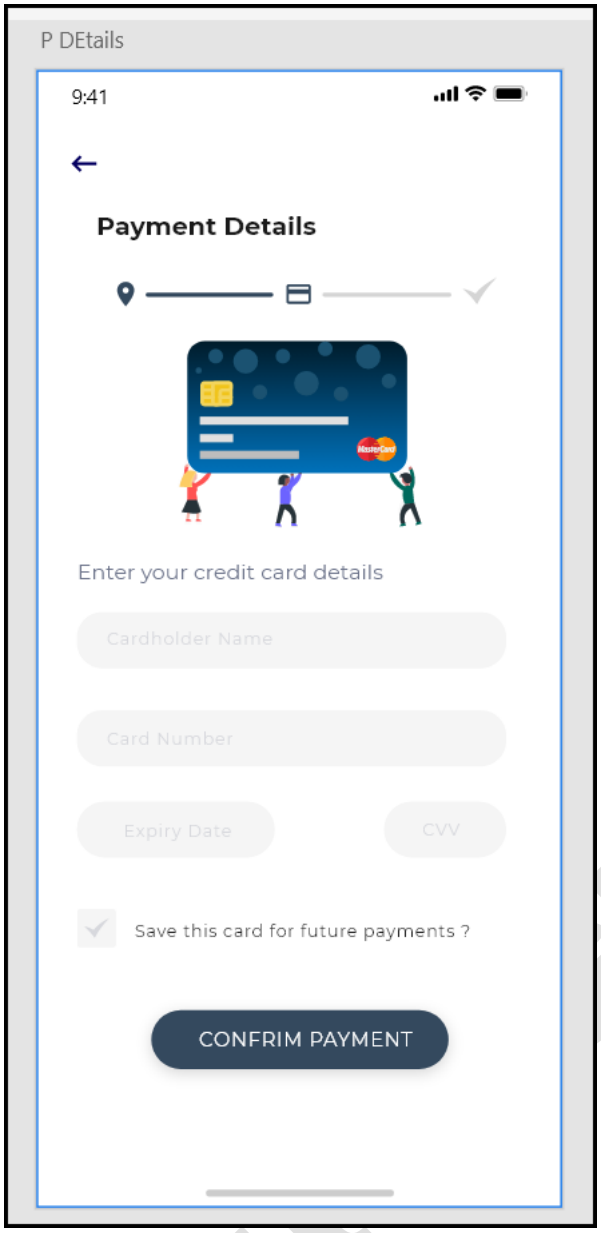
Ejercicio # 1

En esta práctica vamos a continuar con el diseño de la segunda pantalla que tiene relación con los detalles de pago para la aplicación en Flutter.

Detalles de pago #3

1. **Abra** el mockup **PaymentFlow** en **Adobe XD**, marque la **segunda pantalla**, y **visualícela** con un mayor zoom para realizar un análisis, como se muestra a continuación.

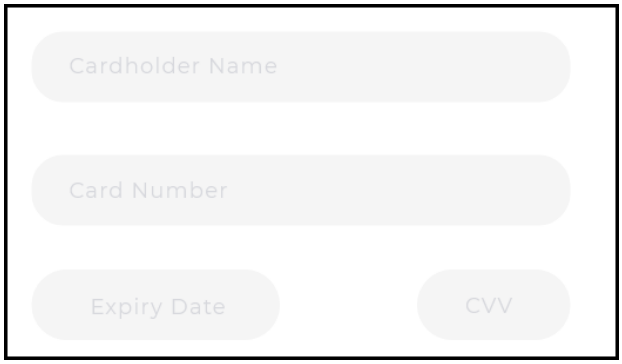




Análisis:

Los siguiente a diseñar para esta práctica, es el espacio donde van los componentes para el **ingreso de datos**.

- Controles para ingresar los datos: **Cardholder Name**, **Card Number**, **Expiry Date**, **CVV**.



2. Para el **diseño** de esta sección de la **UI** se pueden utilizar los siguientes **widgets**, 4 **TextField** con **bordes redondeados** y que además **muestren los textos** con un **color** específico dentro de los widgets de tipo **TextField**.

3. Por lo tanto, abra el archivo **payment_method_screen.dart**, antes el **LIstView**, lo primero que hay que hacer es agregar un **SizeBox** con un **alto** para situar un **espacio** que separe el último **Text** de la práctica anterior y el primer **TextField** que se va a añadir más adelante. El **código** necesario para esto se muestra a continuación.

```
payment_details_screen.dart M
56                    ), // Container
57                  ), // Expanded
58                  const SizedBox(width: 10),
59                  Image.asset('assets/check_mark.png'),
60                  const SizedBox(width: 10),
61                ],
62              ), // Row
63              const SizedBox(height: 10),
64              Image.asset('assets/card_persons.png', height: 125),
65
66              const SizedBox(height: 15),
67              const Text('Ingrese los datos de su tarjeta de crédito',
68                  style: TextStyle(
69                    fontWeight: FontWeight.w300,
70                    fontSize: 15,
71                    color: Color(0XFF6C738A),
72                  )), // TextStyle // Text
73
74
75              const SizedBox(height: 15,)
76
77            ],
78          ), // ListView      You, 1 second ago • Uncommitted changes
79        )), // Padding // Form
80    ); // Scaffold
81  }
```

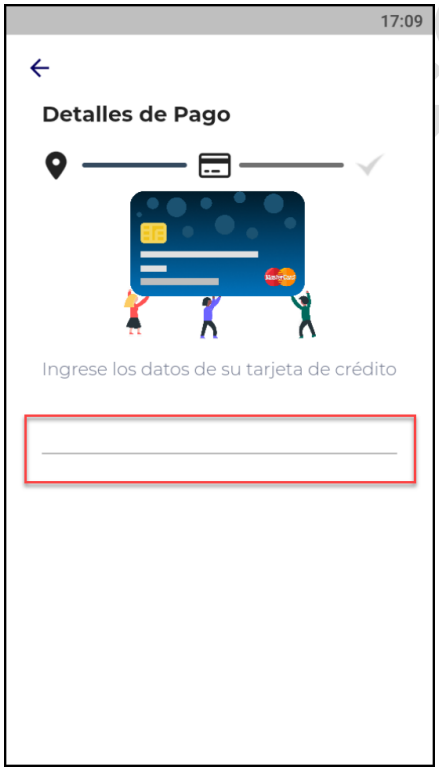
4. Ahora **después** del **SizeBox** agregue un widget/clase **TextFormField**. Este widget además de permitir el **ingreso de texto** (igual que el widget **TextField**), también permite realizar la **validación del texto** ingresado en el componente.

```
payment_details_screen.dart M
                        ), // Expanded
                        const SizedBox(width: 10),
                        Image.asset('assets/check_mark.png'),
                        const SizedBox(width: 10),
                      ],
                    ), // Row
                    const SizedBox(height: 10),
                    Image.asset('assets/card_persons.png', height: 125),
                    const SizedBox(height: 15),
                    const Text('Ingrese los datos de su tarjeta de crédito',
                        style: TextStyle(
                          fontWeight: FontWeight.w300,
                          fontSize: 15,
                          color: Color(0XFF6C738A),
                        )), // TextStyle // Text
                    const SizedBox(height: 15),

                    TextFormField(),

                  ],
                ), // ListView
              )), // Padding // Form
      ); // Scaffold
    }
}       You, last month • Terminado los metodos de pago
```

5. Al **ejecutar** el proyecto el wiget **TextFormField** por omisión muestra la siguiente **UI**.



6. Para que el **TextFormField** tenga la **UI similar** a la que se muestra en el **mockup**, es necesario **personalizarlo**, en el siguiente código se muestra **un ejemplo** de cómo hacerlo con los widgets/clases **TextStyle** (estilo) y **InputDecoration** (decoración).

payment_details_screen.dart U

```
                ), // Text
                const SizedBox(
                  height: 15,
                ), // SizedBox

                TextFormField(
                  style: const TextStyle(
                    color: Colors.black,
                    fontSize: 12,
                  ), // TextStyle
                  decoration: InputDecoration(
                    fillColor: const Color(0xffF5F5F5),
                    filled: true,
                    hintText: 'Nombre',
                    border: OutlineInputBorder(
                      borderRadius: BorderRadius.circular(25),
                      borderSide: BorderSide.none,
                    ), // OutlineInputBorder
                    contentPadding: const EdgeInsets.all(10),
                  ), // InputDecoration
                ), // TextFormField

              ],
            ), // ListView
          ), // Padding
        ), // Form
      ); // Scaffold
    }
  }
```

7. A continuación, una **breve explicación** del código generado anteriormente.

- El widget **InputDecoration** se utiliza para **personalizar** la apariencia y el comportamiento de los campos de entrada de texto, como **TextField** o **TextFormField**. Proporciona una manera de especificar el **aspecto del campo de entrada**, como el texto de etiqueta, el icono, los bordes, los colores de fondo, los márgenes, etc.

- A continuación, se describe algunos datos obtenidos **mockup** utilizando **Adobe XD** y otros utilizados en el código anterior.

PROPIEDAD	VALOR	OBSERVACIÓN
fillColor:	0XFFF5F5F5	Permite cambiar el color de fondo y el hintText.
filled:	true	Establece que el color indicado llene completamente el TextFormField.
hintText:	"Nombre"	Es el texto que se muestra dentro del TextFormFied.
Border:	OutlineInputBorder	Define el aspecto del borde cuando el campo de entrada no tiene el foco. Por defecto, se utiliza OutlineInputBorder, que proporciona un borde rectangular y delgado.

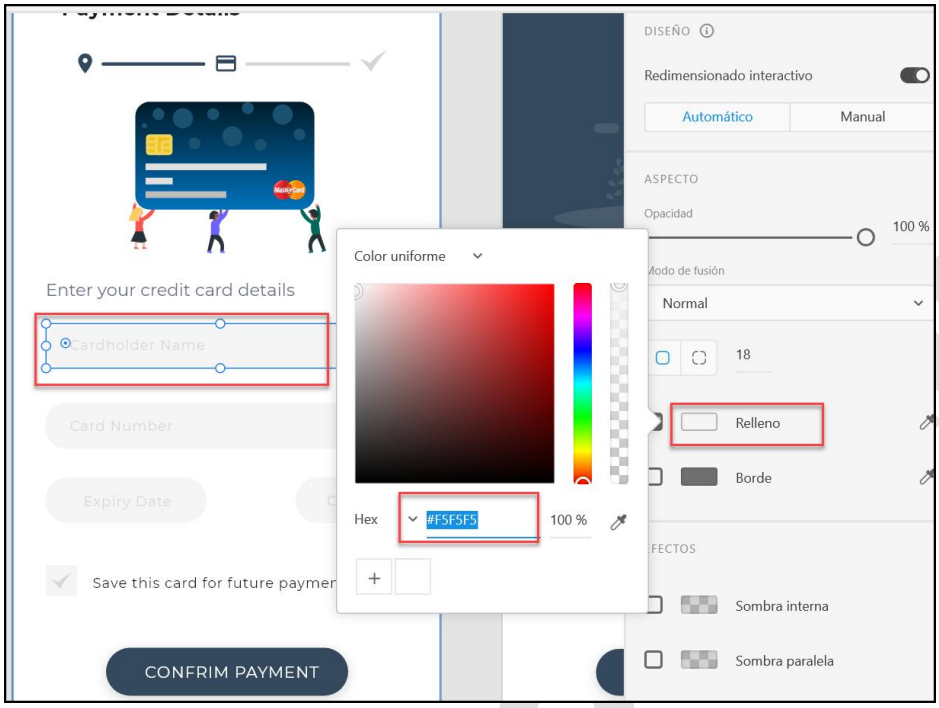
- El widget/clase **OutlineInputBorder** proporciona un borde rectangular alrededor de un campo de entrada de texto, como un **TextField** o un **TextFormField**. Este tipo de borde es comúnmente utilizado en muchas aplicaciones Flutter y es el borde predeterminado cuando no se especifica otro tipo de borde en el widget **InputDecoration** de un campo de entrada.

PROPIEDAD	VALOR	OBSERVACIÓN
borderRadius:	BorderRadius.circular(25)	Permite redondear las esquinas del TextField.
borderSide:	BorderSide.none	Elimina cualquier borde visible alrededor del TextField.
contentPadding:	const EdgeInsets.all(10)	Establece márgenes de 10 píxeles en todas las direcciones (arriba, abajo, izquierda y derecha) alrededor del texto dentro del TextField. Esto crea un espacio uniforme alrededor del texto, lo que puede ser útil para ajustar el espacio entre el texto y los bordes del campo de entrada.

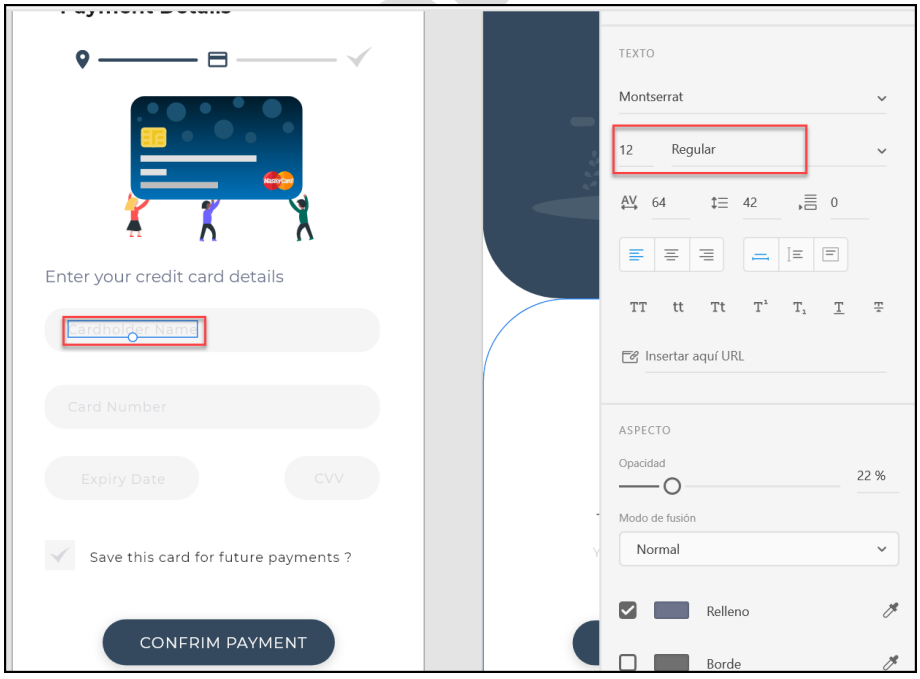
flutter-clay aldás

8. Además, del **mockup** en Adobe XD se obtiene el **color de relleno** del **TextFormField** (propiedad **fillColor**) y el tamaño del texto (propiedad **fontSize**) que va dentro del **TextFormField**.

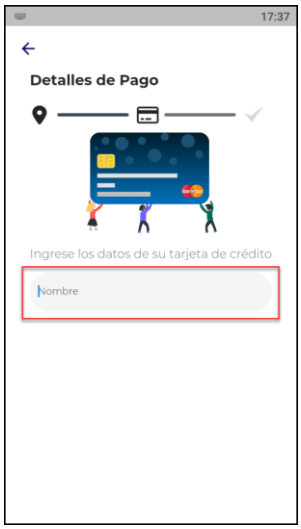
Color de fondo del control: **#F5F5F5**



Tamaño de la **fuente**: **12**

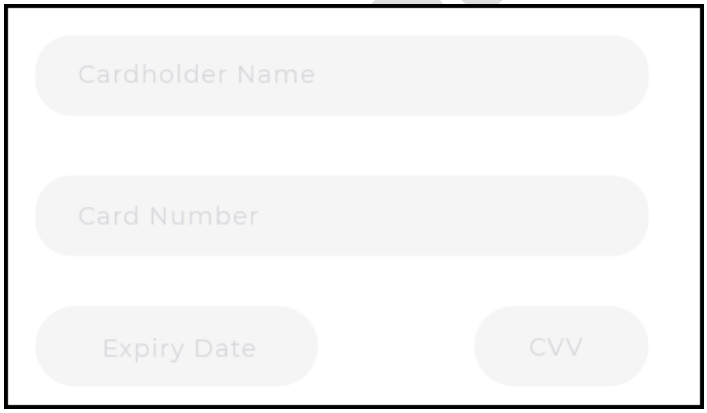


9. **Ejecute** y **prueba** el proyecto.

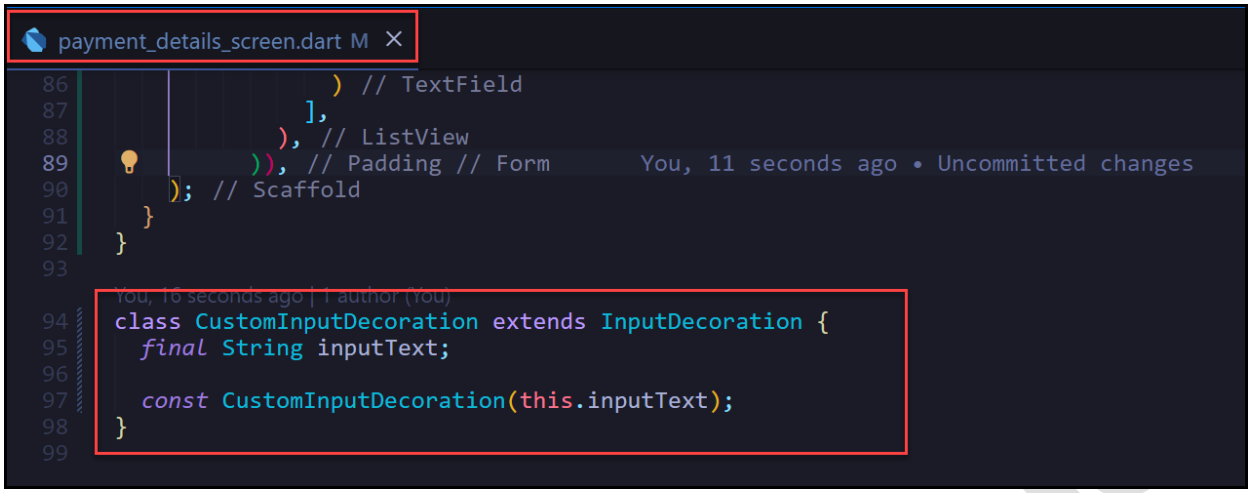


10. Antes de agregar el resto de los campos para el **ingreso de datos**, al analizar el **mockup** se puede observar que el **estilo es el mismo para todos los campos** de tipo **TextFormField**, por lo tanto, lo mejor es **crear una clase** para la **decoración** de forma genérica, y pasar únicamente el valor para la propiedad **hintText** que es lo que cambia entre cada **TextFormField**, y así poder **reutilizar el código tantas veces como sea necesario**.

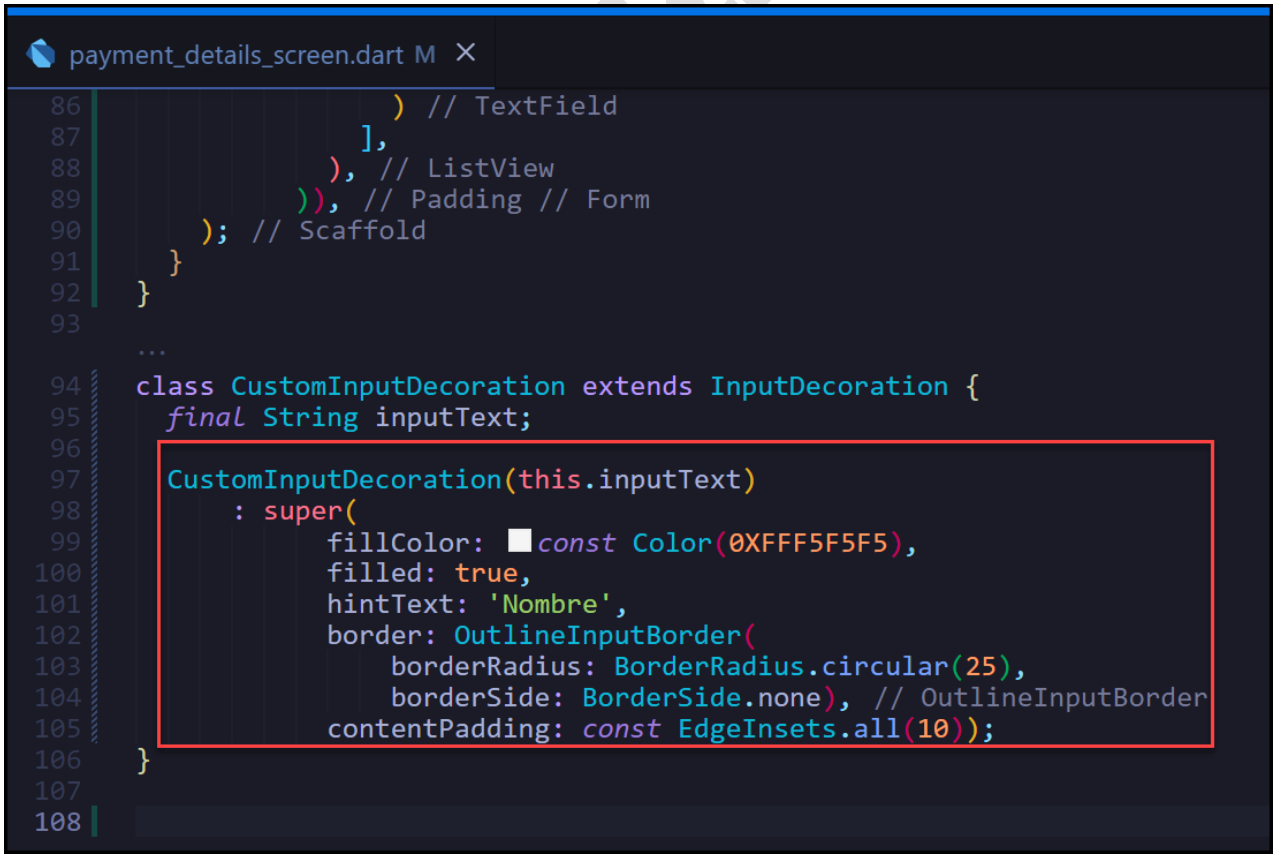
Campos para el ingreso de datos en el **mockup**.



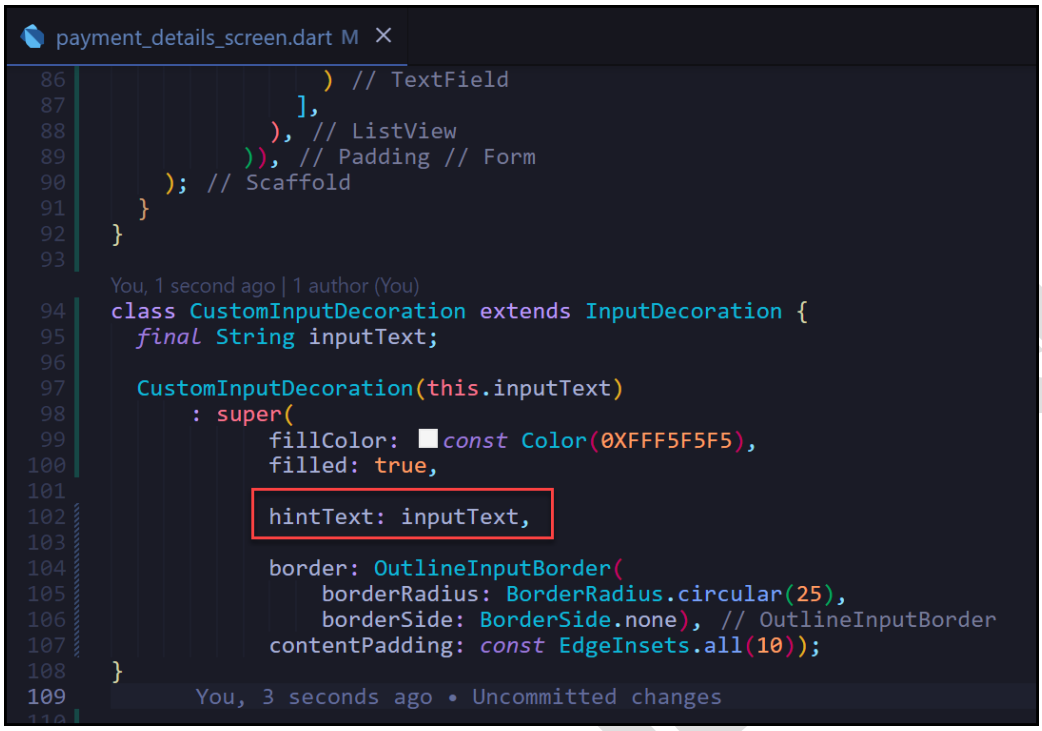
11. Para esto vaya al final del archivo **payment_details_screen.dart**, cree la clase llamada **CustomInputDecoration** y herédela de la clase **InputDecoration**, con un **campo** de tipo **String** llamado **inputText** y su respectivo **constructor** que pase como parámetro esta variable para **inicializarlo**, esto se muestra a continuación.



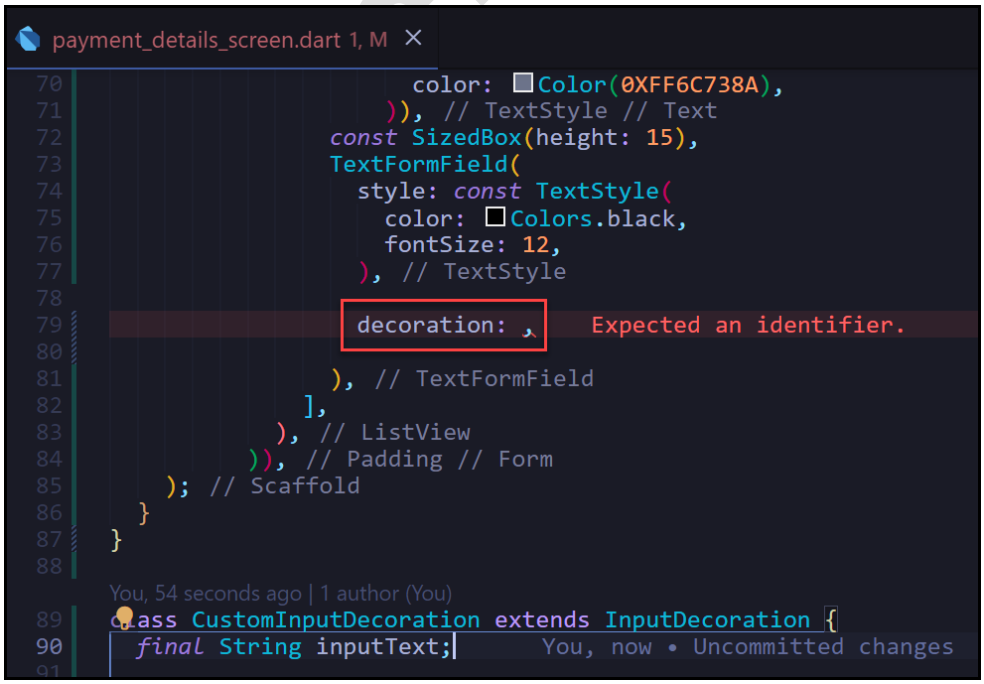
12. Ahora **modifique el constructor** creado, invoque al **constructor** de la clase **padre**, es decir utilizando para esto el **pronombre super**, como parámetros de constructor **super** (constructor de la clase padre) **copiamos los mismos parámetros** usados anteriormente en el **punto 6** para crear un **constructor genérico** para todos los componentes para el **ingreso de datos**, de la siguiente forma.



13. **Modifique** la propiedad **hintText** del componente personalizado para que se asigne el **parámetro inputText** y sea dinámico, según se indica en la siguiente figura.



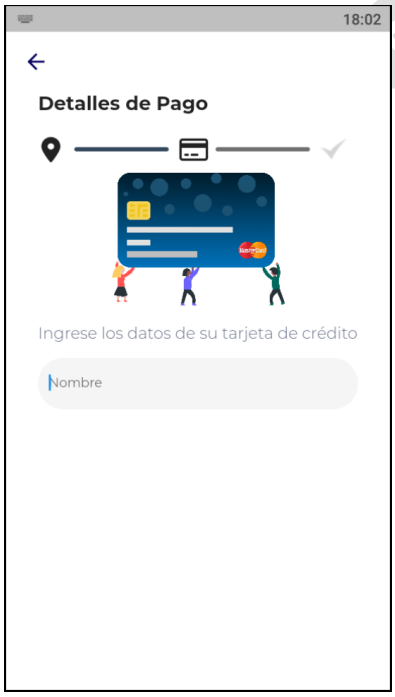
14. **Regrese** al **TextFormField** del punto 6 y **elimine todo** lo que tiene la propiedad **decoration**, esto se muestra en la siguiente figura.



15. En la propiedad **decoration** invoque a la clase **CustomInputDecoration** y pase como parámetro el texto "**Nombre**", para el ingreso del '**Nombre'** en el **componente**, como se muestra a continuación.

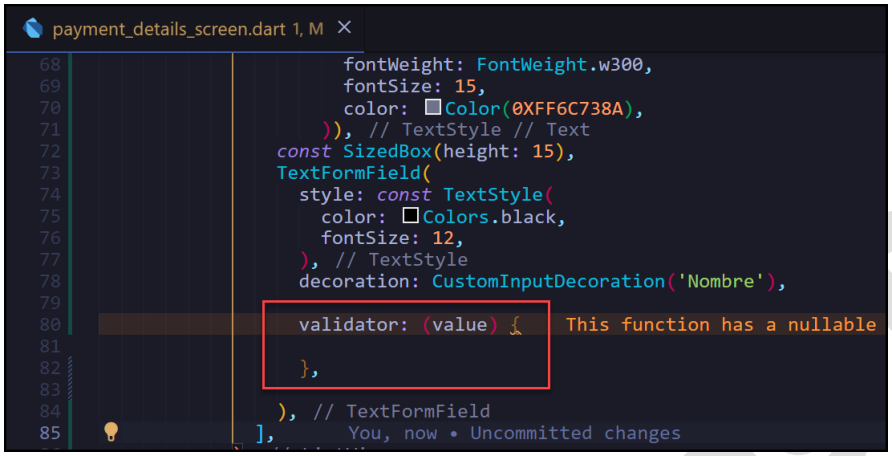
```
payment_details_screen.dart M
65                  const SizedBox(height: 15),
66                  const Text('Ingrese los datos de su tarjeta de crédito',
67                      style: TextStyle(
68                        fontWeight: FontWeight.w300,
69                        fontSize: 15,
70                        color: Color(0XFF6C738A),
71                      )), // TextStyle // Text
72                  const SizedBox(height: 15),
73                  TextFormField(
74                    style: const TextStyle(
75                      color: Colors.black,
76                      fontSize: 12,
77                    ), // TextStyle
78
79                    decoration: CustomInputDecoration('Nombre'),
80
81                  ), // TextFormField
82                ],
83              ), // ListView
84          )), // Padding // Form      You, now • Uncommitted changes
85      ); // Scaffold
86    }
87  }
88
```

16. **Ejecute** y **pruebe** el proyecto. El resultado debe ser el mismo obtenido anteriormente, pero ahora podemos **reutilizar el código**.

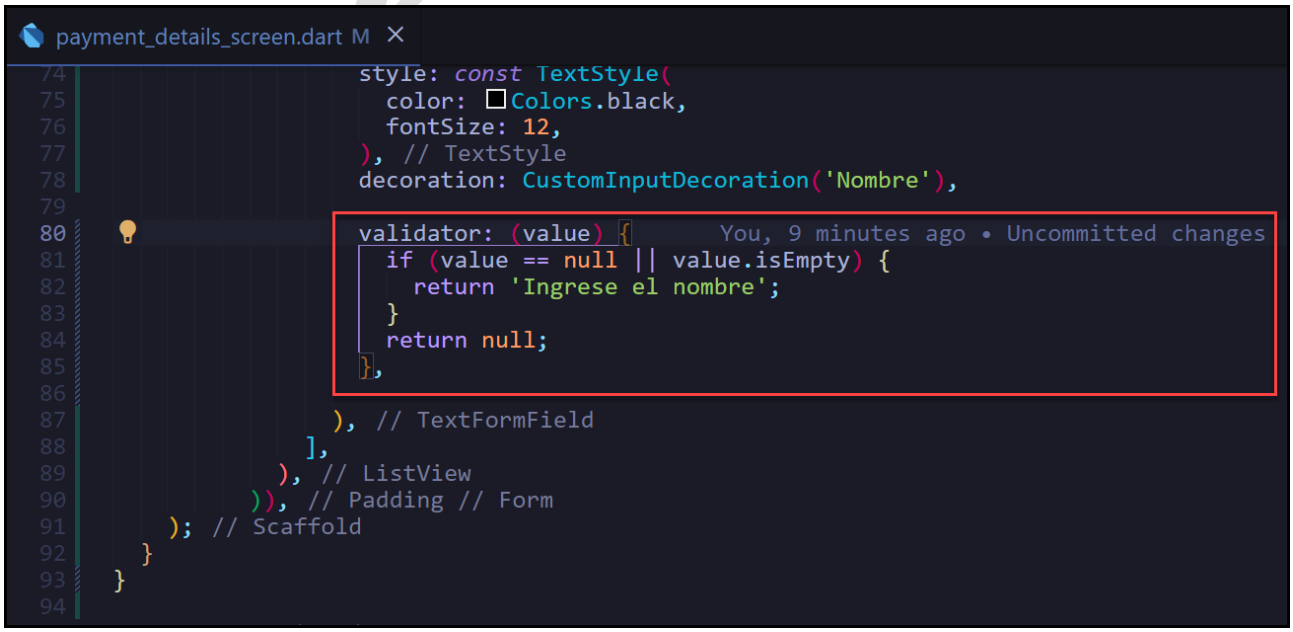


17. Además del formato y la **decoración** agregada al **TextFormField**, vamos también a añadir un **validador** de tal forma que el formulario quede listo para **producción**.

18. Por lo tanto, **luego** de la propiedad **decoration** agregue la propiedad **validator** con un parámetro llamado **value** (aunque el nombre del parámetro puede tener cualquier nombre).



19. **Dentro del bloque** de código anterior (propiedad **validator**) valide el ingreso de datos para que si se pasa como parámetro un **null** o es una **cadena vacía** se muestre un **mensaje**, caso contrario si no hay errores, el validador debe devolver **null**, lo que indica que se pasa la validación.



20. Además de la **validación** necesitamos **guardar el estado** de cada uno de los campos a validar, esto es: el **nombre** del propietario de la tarjeta, el **número** de la tarjeta, **fecha** en la cual se caduca la tarjeta de crédito, y el **código** de seguridad de la tarjeta. Para esto en la clase **_PaymentDetailsScreenState** cree las siguientes variables e inicialícelas.



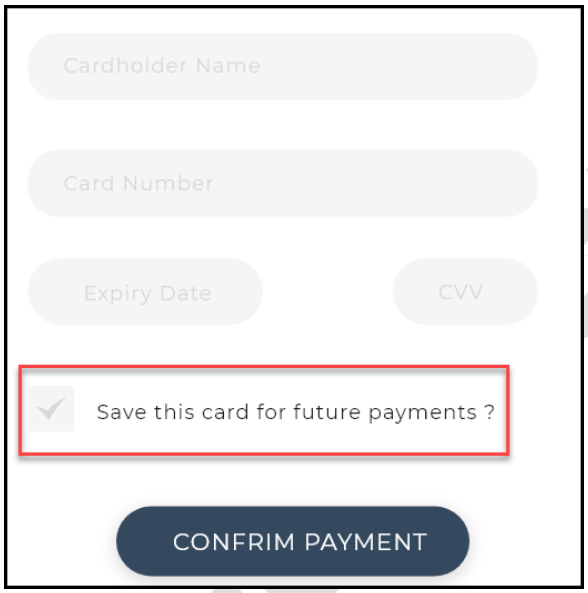
21. Además, **cree** variables de tipo **FocusNode** para establecer el **poner el foco** dentro un campo de ingreso de datos.



Nota	• Late: Es un modificador que, en esencia, permite **diferir** la **inicialización** de la variable. Este modificador se usa en situaciones en las que el valor de la variable **no está disponible en el momento de su declaración,** pero se **garantiza** que estará disponible antes de que se **intente acceder a ella.** Un uso común de **late** es cuando se trabaja con variables que **se inicializarán en un constructor** o en algún **otro lugar del código,** pero **no necesariamente en el momento de la declaración.**

22. Ahora **declare** una variable que se **utilizará** para saber si se desea **guardar los valores ingresados en el formulario**, para el ejemplo esta variable se utilizaría cuando se seleccione el **checkBox** con el texto "**Save this card for future payments** ?. Es decir, para que se guarde los datos ingresados para compras futuras.

En el **mockup** tendría la siguiente **UI**.



Cree la **variable** para guardar los **datos ingresados**.

```dart
payment_details_screen.dart M
11    @override
12    State<PaymentDetailsScreen> createState() {
13      return _PaymentDetailsScreenState();
14    }
15  }
16
    You, 33 seconds ago | 1 author (You)
17  class _PaymentDetailsScreenState extends State<PaymentDetailsScreen> {
18    final _formKey = GlobalKey<FormState>();
19
20    String name = '';
21    String cardNumber = '';
22    String expiryDate = '';
23    String cvvCode = '';
24
25    late FocusNode cardNumberFocus;       You, 11 seconds ago • Uncommitted changes
26    late FocusNode expiryDateFocus;
27    late FocusNode cvvCodeFocus;
28
29    final bool savedCard = false;
30
31    @override
32    Widget build(BuildContext context) {
33      return Scaffold(
34        appBar: AppBar(),
35        body: Form(
```

23. Vaya al final del evento **build** es decir antes del final de la clase **_PaymentDetailsScreenState**, sobrescriba el evento **initState** de la clase e **inicialice el estado** de los objetos de tipo **FocusNode**.

```dart
payment_details_screen.dart 1, M
130       ); // Scaffold
131     }
132
133     @override
134     void initState() {
135       super.initState();
136
137       cardNumberFocus = FocusNode();
138       expiryDateFocus = FocusNode();
139       cvvCodeFocus = FocusNode();
140     }
141
142   }
143
    You, 6 days ago | 1 author (You)
144 class CustomInputDecoration extends InputDecoration {
145   final String inputText;
146
147   CustomInputDecoration(this.inputText)
```

24. Es muy **importante** establecer que cuando **se destruya** el objeto que representa la pantalla, es decir la clase **PaymentDetailsScreen** que es de tipo **StatefulWidget**, también se destruya los objetos de tipo **FocusNode**, por lo tanto, para esto **sobrescriba** el método **dispose** y en este **destruya** los objetos de tipo **FocusNode**.



25. **Regrese** a la propiedad **validator** del **TextFormField** y luego de esta **añada** la propiedad **onSave**, dentro de este bloque de código **guarde el estado** del campo donde se ingresa el **nombre del propietario** de la tarjeta para que se **conserve el valor ingresado en el TextFormField** para la próxima ocasión que se utilice el formulario.



26. Luego del final del **TextFormField** agregue un **espacio** con **SizeBox** para luego agregar el próximo campo para el ingreso de datos.



27. **Seleccione** el widget **TextFormField** y todo su **contenido**, **copie** y **pegue** luego del widget **SizeBox** creado en el punto anterior.

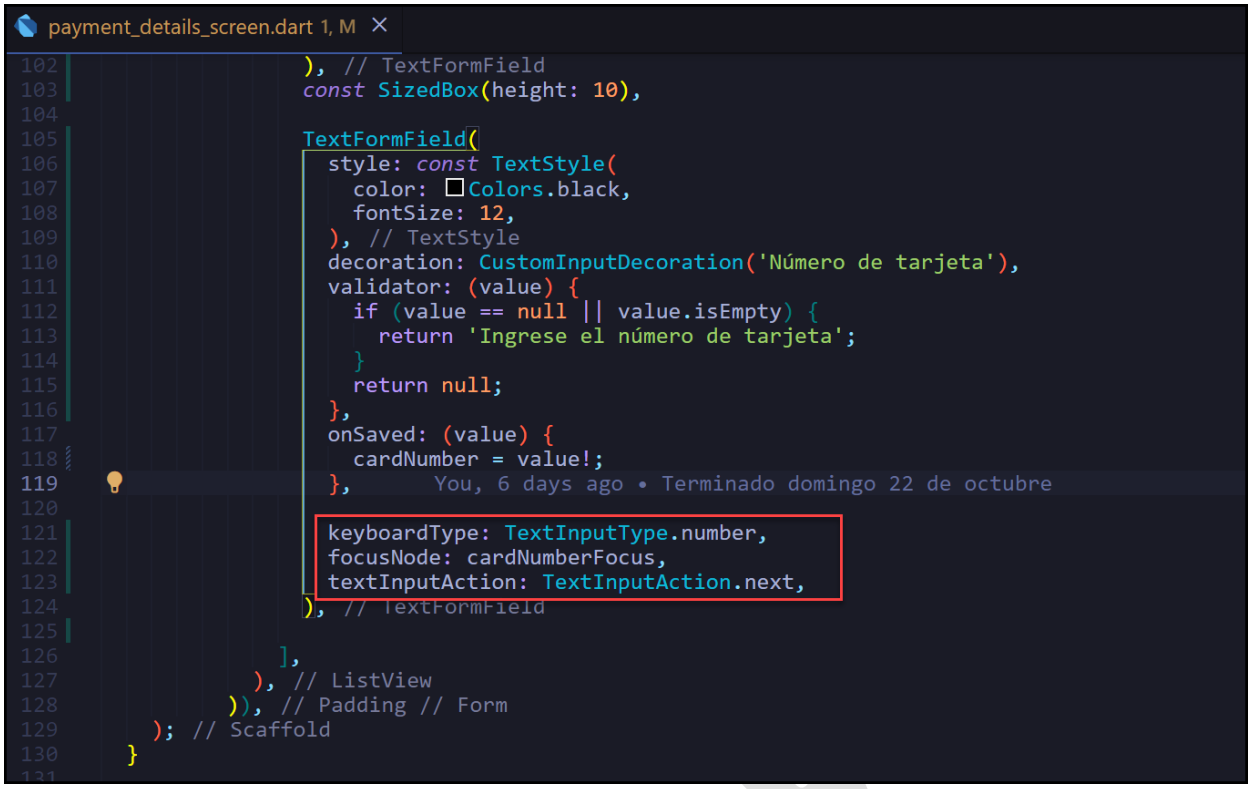


28. En el anterior **código** copiado **modifique** el **texto** del **parámetro** del widget **CustomInputDecoration**, cambie el **texto** que regresa la instrucción **return**, además modifique la propiedad **onSaved** para que **guarde el dato ingresado** en el **TextFormField** en la variable **cardNumber**, como se muestra a continuación.

payment_details_screen.dart 1, M

```
97                  return null;
98                },
99                onSaved: (value) {
100                 name = value!;      You, 37 seconds ago • Uncommitted changes
101               },
102             ), // TextFormField
103             const SizedBox(height: 10),
104
105             TextFormField(
106               style: const TextStyle(
107                 color: Colors.black,
108                 fontSize: 12,
109               ), // TextStyle
110
111               decoration: CustomInputDecoration('Número de tarjeta'),
112
113               validator: (value) {
114                 if (value == null || value.isEmpty) {
115
116                   return 'Ingrese el número de tarjeta';
117
118                 }
119                 return null;
120               },
121               onSaved: (value) {
122
123                 cardNumber = value!;
124
125               },
126
127             ), // TextFormField
```

29. En el anterior campo de **ingreso de datos** hay que realizar algunos cambios. Primero los datos que se desean guardar ya no son de tipo **texto** (String) sino de tipo **número** (**number**) puesto que solo se va a ingresar números (para esto se utiliza la clase **keyboardType**) luego el foco de este **TextFormField** va a ser controlado por el objeto declarado anteriormente (propiedad **cardNumberFocus**) y establecemos la **acción** (**next**) para que permita ir al siguiente campo con el teclado de dispositivo móvil. El código necesario se muestra a continuación.



30. Ahora hay que adicionar una **máscara** para el **ingreso los datos** para que tenga el **formato** de ingreso que se hace en una **tarjeta de crédito**, para esto al inicio del archivo **payment_details_screen.dart** importe el paquete **mask_text_input_formatter**, el cual permite poner máscaras en el ingreso de los datos.



31. Este paquete esta **referenciado** en al archivo **pubspec.yaml** en la sección **dependencies** porque es un paquete externo al **SDK** de Flutter.

payment_details_screen.dart 1, U | pubspec.yaml M

```
32      sdk: flutter
33
34
35    # The following adds the Cupertino Icons font to your applicat
36    # Use with the CupertinoIcons class for iOS style icons.
37    cupertino_icons: ^1.0.6
38    font_awesome_flutter: ^10.6.0
39    mask_text_input_formatter: ^2.7.0
40
41  dev_dependencies:
42    flutter_test:
43      sdk: flutter
44        You, 4 days ago • Terminado practica 002
45    # The "flutter_lints" package below contains a set of recommend
46    # encourage good coding practices. The lint set provided by the
47    # activated in the `analysis_options.yaml` file located at the
48    # package. See that file for information about deactivating spe
49    # rules and activating additional ones.
50    flutter_lints: ^2.0.0
51
```

32. Ahora **agregue** al segundo **TextFormField** la máscara para **formatear** el ingreso de los datos para el **número de tarjeta de crédito**.

payment_details_screen.dart M

```
102                  ), // TextFormField
103                  const SizedBox(height: 10),
104
105                  TextFormField(      You, now • Uncommitted changes
106                    style: const TextStyle(
107                      color: Colors.black,
108                      fontSize: 12,
109                    ), // TextStyle
110                    decoration: CustomInputDecoration('Número de tarjeta'),
111                    validator: (value) {
112                      if (value == null || value.isEmpty) {
113                        return 'Ingrese el número de tarjeta';
114                      }
115                      return null;
116                    },
117                    onSaved: (value) {
118                      cardNumber = value!;
119                    },
120
121                    inputFormatters: <TextInputFormatter>[
122                      MaskTextInputFormatter(mask: '####-####-####-####'),
123                    ], // <TextInputFormatter>[]
124
125                    keyboardType: TextInputType.number,
126                    focusNode: cardNumberFocus,
127                    textInputAction: TextInputAction.next,
128                  ), // TextFormField
129
130                ],
131              ), // ListView
132            )), // Padding // Form
133        ); // Scaffold
```

- La propiedad **inputFormatters** se lo puede agregar en cualquier parte donde están las propiedades: **keyboardType**, **focusNode**, **textInputAction**, para el ejemplo se lo ha agregado en **primer lugar** luego de la propiedad **onSaved**. El **caracter #** significa que en esa posición debe ser ingresado un número y el **carácter –** es un separador que es parte del formato de ingreso de los datos para la tarjeta.

33. **Ejecute** y **pruebe** el proyecto.



34. Ahora vamos a establecer el **foco** en los controles **TextFormField**. Por lo tanto, en el primer **TextFormField** puede ser luego de la propiedad **onSaved** agregue la **propiedad onEditingComplete**, llame al método con el nombre **requestFocus**, digite el siguiente código.

```
payment_details_screen.dart 1, U
101            ), // SizedBox
102            TextFormField(
103              style: const TextStyle(
104                color: Colors.black,
105                fontSize: 12,
106              ), // TextStyle
107              decoration: CustomInputDecoration('Nombre'),
108              validator: (value) {
109                if (value == null || value.isEmpty) {
110                  return 'Ingrese el nombre';
111                }
112                return null;
113              },
114              onSaved: (value) {
115                name = value!;
116              },
117
118              onEditingComplete: () => requestFocus(cardNumberFocus),    The method 'requestFocus'
119
120            ), // TextFormField
121            const SizedBox(height: 10),
122            TextFormField(
123              style: const TextStyle(
```

35. **Cree** el método **requestFocus** al final de la clase, el cual permite **poner el foco** en el **TextFormField** en el campo **cardNumber**, este método se ejecuta cuando se complete la edición en el componente (**onEditingComplete**).

payment_details_screen.dart U

```
154       expiryDateFocus = FocusNode();
155       ccvCodeFocus = FocusNode();
156     }
157
158   @override
159     void dispose() {
160       super.dispose();
161
162       cardNumberFocus.dispose();
163       expiryDateFocus.dispose();
164       ccvCodeFocus.dispose();
165     }
166
167     void requestFocus(FocusNode focusNode) {
168       FocusScope.of(context).requestFocus(focusNode);
169     }
170
171   }
172
173   class CustomInputDecoration extends InputDecoration {
174     final String inputText;
175
176     CustomInputDecoration(this.inputText)
177         : super(
```

36. En el código anterior se llama a **FocusScope** desde el **contexto** (context) y solicitamos el foco para el parámetro que se le pasa al método, es decir para los datos que se ingresen dentro de uno de los **TextFormField** donde se llame al método **requestFocus**.

37. El objeto **FocusNode** permite que el usuario pueda navegar entre los campos de una forma más fácil, por lo tanto, **agregue** al primer **TextFormField** también la propiedad **textInputAction**, como se muestra a continuación.

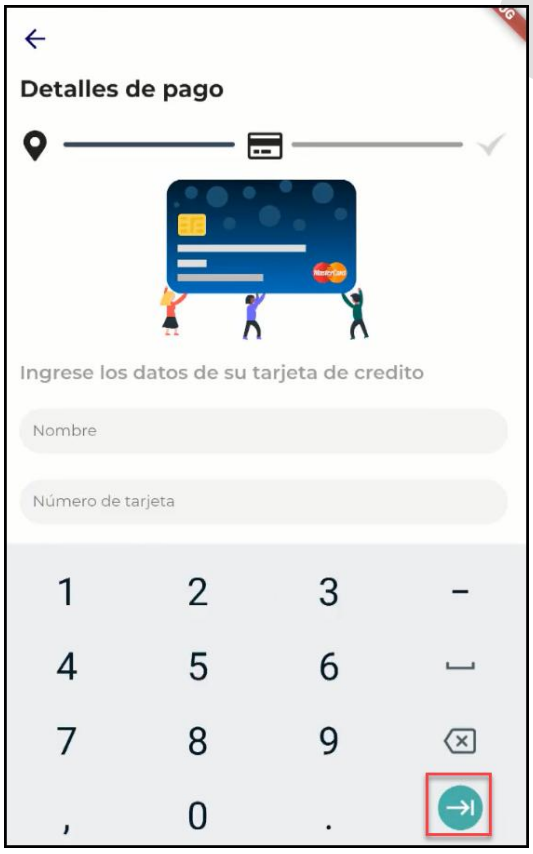
payment_details_screen.dart M

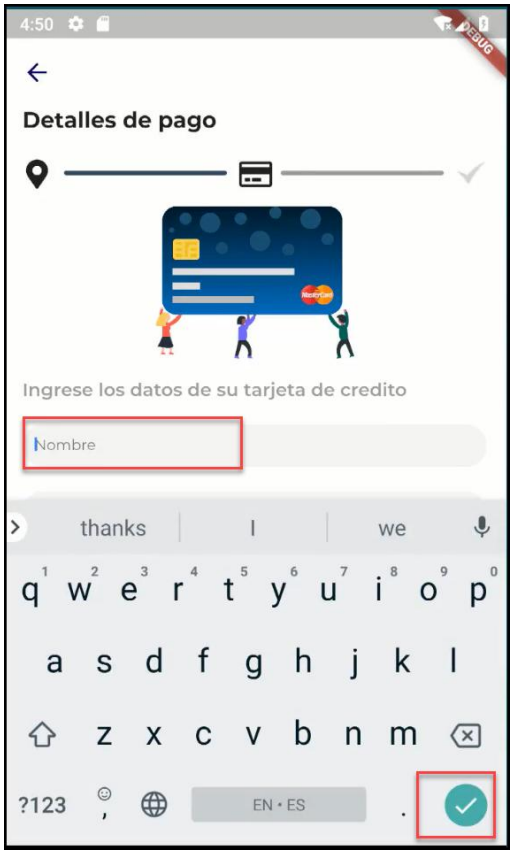
```
                fontSize: 15,
                color: Color(0XFF6C738A),
              )), // TextStyle // Text
          const SizedBox(height: 15),
          TextFormField(      You, 6 days ago • Terminado domingo 22 de octubre
            style: const TextStyle(
              color: Colors.black,
              fontSize: 12,
            ), // TextStyle
            decoration: CustomInputDecoration('Nombre'),
            validator: (value) {
              if (value == null || value.isEmpty) {
                return 'Ingrese el nombre';
              }
              return null;
            },
            onSaved: (value) {
              name = value!;
            },
            onEditingComplete: () => requestFocus(cardNumberFocus),

            textInputAction: TextInputAction.next,

          ), // TextFormField
          const SizedBox(height: 10),
          TextFormField(
```

38. **Ejecute** y **pruebe** el proyecto.





Lecturas útiles para revisar y profundizar en los temas abordados en esta práctica.

TextFormField class

https://api.flutter.dev/flutter/material/TextFormField-class.html

TextStyle class

https://api.flutter.dev/flutter/painting/TextStyle-class.html

InputDecoration class

https://api.flutter.dev/flutter/material/InputDecoration-class.html

OutlineInputBorder class

https://api.flutter.dev/flutter/material/OutlineInputBorder-class.html

TextInputType class

https://api.flutter.dev/flutter/services/TextInputType-class.html

TextInputAction enum

https://api.flutter.dev/flutter/services/TextInputAction.html

Input formatters in Textfield Flutter

https://blog.nonstopio.com/input-formatters-in-textfield-flutter-22f986f3646b

Flutter Package Provides Textinputformatter For TextField/TextFormField

flutterClay Aldás

https://morioh.com/a/20b9dbf5ee7e/flutter-package-provides-textinputformatter-for-textfieldtextformfiel

FlutterClay Aldás